

In [ ]: Neste jupyter notebook, constam os fontes utilizados em Fundamentos de Py dicas de ouro, engenharia de softwares, testes de software, IA (ML) e Ciê na Linguagem de Programação Python3, alcançando novos patamares em TIC.

```
In [1]: # Fundamentos de Python - Ambiente Jupyter Notebook
...
A ferramenta estará disponível em https://www.python.org/downloads/, ou a
--> no ambiente Windows, pode-se fazer o "Download Windows installer (64-
que ele fará a instalação.
--> no linux é nativo. caso sua distro não tenha, para os derivados debia
--> para os demais casos, seguir as orientações do site suprarreferido.
-> python online: https://www.onlinegdb.com/online_python_compiler ou htt
...
print("Exemplo de bloco comentado com 1 e várias linhas!")
```

Exemplo de bloco comentado com 1 e várias linhas!

```
In [2]: # Nosso primeiro programa em Python
print("olá mundo")
```

olá mundo

```
In [3]: # Literais de string
print("Usando aspas duplas")
print('Usando aspas simples')
print("Escrevendo a palavra 'Python' entre aspas")
print("Incorporando uma\nquebra de linha com \\n")
print("""Incorporando uma quebra de de linha com três aspas duplas""")
```

Usando aspas duplas  
Usando aspas simples  
Escrevendo a palavra 'Python' entre aspas  
Incorporando uma  
quebra de linha com \n  
Incorporando uma quebra de de linha com três aspas duplas

```
In [4]: # Principais funções simples e essenciais do Python, divididas por catego
# vamos iniciar a conhecida função print(), já utilizadas acima
```

```
In [5]: # abs(x): Retorna o valor absoluto (positivo) de um número.
print(abs(-5)) # Saída: 5
```

5

```
In [6]: # min(): Retorna o menor valor entre os argumentos ou de uma lista.
print(min(3, 1, 4, 2)) # Saída: 1
```

1

```
In [7]: # max(): Retorna o maior valor entre os argumentos ou de uma lista.
print(max([10, 20, 30])) # Saída: 30
```

30

```
In [8]: # pow(base, exp): Eleva a base ao expoente (igual a base exp).
print(pow(2, 3)) # Saída: 8
```

8

```
In [9]: # round(numero, digitos): Arredonda um número para uma quantidade de cas
print(round(3.14159, 2)) # Saída: 3.14
```

3.14

```
In [10]: # sum(lista): Soma todos os elementos de uma lista ou tupla numérica.
print(sum([1, 2, 3, 4])) # Saída: 10
```

10

```
In [11]: # 2. Funções de Conversão de Tipo (Casting)
```

```
In [12]: # Exemplos
valor = "10" #usadono exemplo
#valor
print (int(valor),float(valor), str(valor), bool(valor)) #converte o valo
```

10 10.0 10 True

```
In [13]: # 3. Funções de Interação e Informação
```

```
In [14]: # print(): Exibe uma mensagem ou resultado na tela.
print("Olá, mundo de aprendizado --> " + 3 * " Python3!" )
```

Olá, mundo de aprendizado --> Python3! Python3! Python3!

```
In [ ]: # input(): Recebe um dado digitado pelo usuário (sempre retorna o dado co
nome = input("Digite seu nome: ")
print (f"Seja bem vindo Sr(a) {nome}! Obrigado pelo seu cadastro")
```

```
In [ ]: # len(): Retorna o tamanho (quantidade de caracteres de um texto ou itens
print(len("Python")) # Saída: 6 (letras)
```

```
In [ ]: # type(): Revela o tipo de dado de uma variável.
print(type(3.14)) # Saída: <class 'float'>
```

```
In [ ]: # 4. Funções para Sequências e Listas (intermediário / avançado)
```

```
In [ ]: # range(inicio, fim): Cria uma sequência de números (muito usada em loop
# Gera números de 0 a 4
print(list(range(5))) # Saída: [0, 1, 2, 3, 4]
```

```
In [ ]: # sorted(lista): Retorna uma nova lista com os elementos ordenados.
print(sorted([3, 1, 2])) # Saída: [1, 2, 3]
```

```
In [ ]: # Lista de números desordenados
numeros = [4, 2, 9, 1, 5]

# Ordenando
numeros_ordenados = sorted(numeros)

print("Original:", numeros) # [4, 2, 9, 1, 5] (Não muda!)
print("Ordenada:", numeros_ordenados) # [1, 2, 4, 5, 9]
```

```
In [ ]: letras = ['b', 'd', 'a', 'c']

# Ordenando de trás para frente
letras_reverse = sorted(letras, reverse=True)
```

```
print(letras_reverse) # ['d', 'c', 'b', 'a']
```

```
In [ ]: palavras = ["banana", "uva", "abacaxi", "morango"]

# O Python vai olhar para o tamanho (len) de cada palavra para ordenar
palavras_por_tamanho = sorted(palavras, key=len)

print(palavras_por_tamanho) # ['uva', 'banana', 'abacaxi', 'morango']
```

```
In [ ]: produtos = [
    {"nome": "Celular", "preco": 1500},
    {"nome": "Livro", "preco": 50},
    {"nome": "Fone de Ouvido", "preco": 200}
]

# A chave de ordenação será o valor da propriedade 'preco'
produtos_por_preco = sorted(produtos, key=lambda produto: produto["preco"])

print(produtos_por_preco)
# Resultado: Livro (50), Fone (200), Celular (1500)
```

```
In [4]: # list.sort()
lista_numeros = [3, 1, 2] # passa os valores para a lista
print(lista_numeros) # imprime a lista original

lista_numeros.sort() #modifica a lista original - ordenando-a

print(lista_numeros) # Saída: [1, 2, 3]
```

```
[3, 1, 2]
```

```
[1, 2, 3]
```

```
In [ ]: animais = ["cachorro", "gato", "abelha", "zebra"]

# Ordena em ordem alfabética invertida
animais.sort(reverse=True)

print(animais) # Saída: ['zebra', 'gato', 'cachorro', 'abelha']
```

```
In [ ]: frutas = ["Morango", "banana", "Abacaxi", "uva"]

# Sem o key, "Morango" e "Abacaxi" viriam primeiro por causa da maiúscula
frutas.sort(key=str.lower)

print(frutas) # Saída: ['Abacaxi', 'banana', 'Morango', 'uva']
```

```
In [ ]: # Desafio Python 3 - 1
"""
Faça um pequeno programa em Python3, para atender a um cliente de uma em
Na aplicação, o cliente deve inserir a nota do aluno, para fins de teste
Apliquem os conhecimentos disponível na literatura e material.
"""
```

```
In [20]: print("\n" + "="*50)
print("=== CADASTRO - NOTAS ALUNOS ===")
print("\n" + "="*50)
```

```

nota = float(input("Digite a Nota da Disciplina: ")) #atenção para a conv
if nota >= 7:
    print("Aprovado")
elif nota >= 5:
    print("Recuperação")
else:
    print("Reprovado")

```

=====

=== CADASTRO - NOTAS ALUNOS ===

=====

Reprovado

```

In [ ]: # Desafio Python 3 - 2
"""
Faça um pequeno programa em Python3, para atender a um cliente de uma em
Na aplicação, o cliente deve inserir a nota do aluno, para fins de teste
Aplicuem os conhecimentos disponível na literatura e material.

=====> nesta versão, deverá haver uma função, para o cálculo da média de 2
"""

```

```

In [24]: print("\n" + "="*50)
print("=== CADASTRO - NOTAS ALUNOS ===")
print("="*50)

## aqui, aprendemos a criar função
## (def keyword usada para definir função.Sintaxe)
def calcular_media(n1, n2):
    return (n1 + n2) / 2

nota1 = float(input("Nota 1: "))
nota2 = float(input("Nota 2: "))
media = calcular_media(nota1, nota2)

print("Média:", media)
if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")

```

=====

=== CADASTRO - NOTAS ALUNOS ===

=====

Média: 6.05

Reprovado

```

In [ ]: ### Desafio Python3
"""
==== REQUISITOS =====
Faça um pequeno programa em Python3, para atender a um cliente de uma em
Na aplicação, o cliente deve inserir seu nome e telefone de contato. Após
serão gerados 5 numeros das sorte, que devem ser exibidos, de modo ordena
concatenação do menor numero e o maior numero.

```

Após conclusos, deve ser exibido os dados digitados e uma saudação de " p os números da sorte o telefone de contato e a senha do cliente.

```
'''
print("\n--- DESAFIO PYTHON 3 - 3 === \n")
```

```
In [6]: # Solução proposta
import random

def main():
    print("=== CADASTRO - CRYPTOATIVOS ===")

    # 1. Coleta de dados do cliente
    nome = input("Digite seu nome completo: ").strip()
    telefone = input("Digite seu telefone de contato (com DDD): ").strip()

    # 2. Geração de 5 números da sorte (valores entre 1 e 100, sem repeti
    # Usamos random.sample para garantir que os 5 números sejam distintos
    numeros_da_sorte = random.sample(range(1, 101), 5)

    # Ordena os números em ordem crescente
    numeros_da_sorte.sort()

    # 3. Definição da senha (concatenação do menor e do maior número)
    # Como a lista já está ordenada, o menor é o primeiro [0] e o maior é
    menor_numero = numeros_da_sorte[0]
    maior_numero = numeros_da_sorte[-1]
    senha_acesso = f"{menor_numero}{maior_numero}"

    # Converte os números para uma string bonita para exibição (ex: 12, 2
    numeros_formatados = ", ".join(map(str, numeros_da_sorte))
    #Inserir figuras para usabilidade
    # Usando o código Unicode (substitua o U+ por 000)
    print("Acesse com a sua chave: \U0001F511")
    # 4. Exibição do relatório final e saudação
    print("\n" + "="*50)
    print(f"Prezado(a) Sr(a) {nome}, obrigado pelo seu cadastro!")
    print("="*50)
    print(f"📞 Telefone de contato: {telefone}")
    print(f"🍀 Números da sorte (ordenados): {numeros_formatados}")
    print(f"🔑 Senha de acesso gerada: {senha_acesso}")
    print("="*50)

#print( __name__ )
if __name__ == "__main__":
    main()
```

```
__main__
=== CADASTRO - CRYPTOATIVOS ===
Acesse com a sua chave: 🔑

=====
Prezado(a) Sr(a) dddd, obrigado pelo seu cadastro!
=====
📞 Telefone de contato: 66666666
🍀 Números da sorte (ordenados): 12, 25, 56, 58, 76
🔑 Senha de acesso gerada: 1276
=====
```

```
In [ ]: ##### INTRODUÇÃO AOS FUNDAMENTOS INTERMEDIÁRIOS E AVANÇADOS
# ==== LISTAS [], TUPLAS (), DICIONARIOS [;]
```

```
In [ ]: # ==> Listas (list)
```

```
In [ ]: # Lista de strings
frutas = ["maçã", "banana", "laranja"]

# Lista de números (inteiros e floats)
notas = [7.5, 8.0, 10, 6.5]

# Lista mista (aceita diferentes tipos de dados)
misturado = ["Python", 3, True, 4.5]

# Lista vazia
vazia = []
```

```
In [ ]: # Acessando os elementos
#           0           1           2
frutas = ["maçã", "banana", "laranja"]

print(frutas[0]) # Saída: maçã
print(frutas[2]) # Saída: laranja
print(frutas[-1]) # Saída: laranja (último item)
```

```
In [ ]: # modificando Listas
animais = ["cachorro", "gato", "pássaro"]
animais[1] = "gato de botas"

print(animais) # Saída: ['cachorro', 'gato de botas', 'pássaro']
```

```
In [ ]: # Adicionar itens.
# append(): Adiciona um item ao final da lista.
# insert(): Insere um item em uma posição específica.
num = [1, 2, 3]

num.append(4)
print(num) # Saída: [1, 2, 3, 4]

num.insert(1, 99) # Insere o 99 no índice 1
print(num) # Saída: [1, 99, 2, 3, 4]
```

```
In [ ]: # Remover itens
# remove(): Remove o primeiro item com o valor especificado.
# pop(): Remove e retorna o item de um índice (se não passar o índice,
cores = ["azul", "verde", "vermelho", "verde"]

cores.remove("verde") # Remove apenas o primeiro 'verde'
print(cores) # Saída: ['azul', 'vermelho', 'verde']

ultima_cor = cores.pop() # pilhas (stack), seguindo o princípio LIFO (Last In, First Out)
print(ultima_cor) # Saída: verde
print(cores) # Saída: ['azul', 'vermelho']
```

```
In [ ]: # Ordenação e Tamanho
# len(): Retorna o tamanho (número de elementos) da lista.
# sort(): Ordena a lista (em ordem alfabética ou crescente).
```

```
valores = [5, 2, 9, 1]

print(len(valores)) # Saída: 4

valores.sort()
print(valores)      # Saída: [1, 2, 5, 9]
```

```
In [ ]: #Percorrendo uma Lista (Loop for)
# mais comum: usando o laço for.
linguagens = ["Python", "JavaScript", "C++"]

for lingua in linguagens:
    print(f"Eu programo em {lingua}!")
```

```
In [ ]: # ==> Tuplas (tuples)
# Quando usar Tuplas em vez de Listas?

# Segurança: Para garantir que os dados não sejam alterados por acidente
# Performance: Tuplas são ligeiramente mais rápidas e consomem menos memória
# Chaves de Dicionário: Como são imutáveis, as tuplas podem ser usadas como chaves
```

```
In [ ]: # Tupla vazia
tupla_vazia = ()

# Tupla de strings (mais comum, usando parênteses)
frutas = ("maçã", "banana", "morango")

# Tupla com diferentes tipos de dados (heterogênea)
pessoa = ("Ana", 28, 1.65, True) # Nome, Idade, Altura, Ativo

# Tupla definida sem parênteses (Tuple Packing)
coordenadas = -23.5505, -46.6333 # Latitude e Longitude de SP

# CUIDADO: Tupla com um único elemento precisa de uma vírgula no final!
tupla_unitaria = ("apenas um elemento",) #sem vírgula, será uma string
```

```
In [ ]: # Acessando Elementos (Indexação e Slicing)
# O acesso funciona igual ao de uma lista, utilizando colchetes []
# e índices que começam no 0.
cores = ("vermelho", "verde", "azul", "amarelo")

# Acessando pelo índice
print(cores[0]) # Saída: vermelho
print(cores[-1]) # Saída: amarelo (último elemento)

# Cortando a tupla (Slicing)
print(cores[1:3]) # Saída: ('verde', 'azul')
```

```
In [ ]: # Prova de que é Imutável
# Vamos tentar alterar um valor,
# o Python vai retornar um erro (TypeError):
dimensoes = (1920, 1080)

# Isso vai gerar um erro:
# dimensoes[0] = 1280
```

```
In [ ]: # Desempacotamento de Tuplas (Unpacking)
# Pode-se extrair os valores de uma tupla
# diretamente para variáveis individuais.

produto = ("Televisão", 3500.00, "Eletrônicos")

# Desempacotando
nome, preco, categoria = produto

print(nome)      # Saída: Televisão
print(preco)     # Saída: 3500.0
```

```
In [ ]: # Métodos Úteis com tuplas
# As tuplas são imutáveis, elas possuem
# apenas dois métodos próprios:
numeros = (1, 2, 3, 1, 4, 1, 5)

# .count(x): Conta quantas vezes o
# elemento 'x' aparece
print(numeros.count(1)) # Saída: 3

# .index(x): Retorna o índice da primeira
# aparição do elemento 'x'
print(numeros.index(4)) # Saída: 4
```

```
In [ ]: ### Dicionários
"""
Um dicionário (dict) é uma coleção de dados mutável e ordenada (a partir
Armazena elementos no formato de chave: valor (key: value).
Imagine um dicionário real: você procura por uma palavra (chave) para enc
No Python é a mesma coisa, você usa a chave para acessar o valor rapidame
"""
```

```
In [ ]: # Criando Dicionários
# Os dicionários são definidos utilizando chaves {}
# e os pares são separados por dois pontos :.

# Dicionário vazio
dicionario_vazio = {}

# Dicionário de um produto (Chaves são strings,
# valores são tipos variados)
produto = {
    "nome": "Teclado Mecânico",
    "preco": 299.90,
    "em_estoque": True,
    "cores_disponiveis": ["Preto", "Branco"]
}

# Criando com a função dict()
usuario = dict(nome="Francisco", idade=30, cargo="Prof")
```

```
In [ ]: # Acessando Valores
# Para acessar um valor, você usa o nome do dicionário
# seguido da chave entre colchetes [] ou o método .get().
carro = {
    "marca": "Renault",
    "modelo": "Logan",
```

```

    "ano": 2015
}

# Forma direta (Gera erro se a chave não existir)
print(carro["modelo"]) # Saída: Mustang

# Usando o método .get() (Mais seguro:
# retorna None ou um padrão se a chave não existir)
print(carro.get("ano")) # Saída: 2023
print(carro.get("cor", "Não informada"))
# Saída: Não informada

```

```

In [ ]: # Adicionando e Modificando Elementos
# Como os dicionários são mutáveis, você pode alterar
# valores existentes ou adicionar novas chaves diretamente.
aluno = {
    "nome": "Pedro",
    "curso": "Engenharia de Dados"
}

# Modificando um valor existente
aluno["curso"] = "Ciência da Computação"

# Adicionando uma nova chave-valor
aluno["nota_final"] = 9.5

print(aluno)
# Saída: {'nome': 'Pedro', 'curso': 'Ciência da Computação', 'nota_final'

```

```

In [ ]: # Removendo Elementos
# Existem algumas formas de deletar dados de um dicionário:
smartphone = {
    "marca": "Apple",
    "modelo": "iPhone 15",
    "cor": "Azul",
    "bateria": "100%"
}

# 1. Método .pop(): remove a chave e retorna o valor dela
cor_removida = smartphone.pop("cor")

# 2. Palavra-chave 'del': remove direto e não retorna nada
del smartphone["bateria"]

# 3. Método .popitem(): remove o último elemento inserido
smartphone.popitem()

```

```

In [ ]: # Iterando (Navegando) em Dicionários
# Você pode percorrer apenas as chaves,
# apenas os valores ou ambos ao mesmo tempo
# usando loops for.
capital_estados = {
    "SP": "São Paulo",
    "RJ": "Rio de Janeiro",
    "MG": "Belo Horizonte"
}

# Iterando pelas Chaves (Padrão)
for sigla in capital_estados:

```

```

    print(sigla) # Saída: SP, RJ, MG

# Iterando pelos Valores (.values())
for cidade in capital_estados.values():
    print(cidade) # Saída: São Paulo, Rio de Janeiro...

# Iterando por ambos (.items()) - Muito utilizado!
for sigla, cidade in capital_estados.items():
    print(f"A capital de {sigla} é {cidade}.")

```

In [ ]: # Métodos Úteis - trunfos úteis - pcp nas fases de verificação e validação

```

    # dicionario.keys(): Retorna uma lista com todas as chaves.

    # dicionario.values(): Retorna uma lista com todos os valores.

    #dicionario.clear(): Limpa o dicionário inteiro.

    # dicionario.update(outro_dict): Atualiza o dicionário com pares de o

```

In [7]: #===== EXTRAS --> integração tuplas (imutável) + dicionários (mutável)

```

# 1. Usando TUPLAS para chaves e dados que NÃO mudam (Segurança e Performance)
# Uma tupla como chave do dicionário (ID do produto, Categoria)
# Tuplas também controlam o tamanho da caixa do produto (Largura, Altura,
estoque = {
    (101, "Consoles"): {
        "nome": "PlayStation 5",
        "preco": 4500.00,
        "dimensoes": (39, 10, 26)
    },
    (102, "Jogos"): {
        "nome": "Elden Ring",
        "preco": 299.90,
        "dimensoes": (17, 13, 1)
    }
}

# 2. Usando o .get() para evitar erros se o produto não existir
# Procurando um ID que não existe no estoque
produto_fantasma = estoque.get((999, "Acessórios"), "Produto não encontrado")
print(f"Busca segura: {produto_fantasma}")
# Saída: Busca segura: Produto não encontrado!

# 3. Desempacotamento (Unpacking) ao navegar pelo dicionário com .items()
print("\n--- Relatório de Envios (Usando Desempacotamento) ---")
for (id_prod, categoria), dados in estoque.items():
    # Desempacotando a tupla de dimensões que estava guardada no dicionário
    largura, altura, prof = dados["dimensoes"]

    print(f"Produto: {dados['nome']} | Categoria: {categoria}")
    print(f"Dimensões para frete: {largura}cm x {altura}cm x {prof}cm")

# 4. Atualizando o dicionário em lote com o método .update()
# Chegou um lote novo com atualização de preço e um novo produto
novos_dados = {
    (101, "Consoles"): {
        "nome": "PlayStation 5 Slim", # Atualizou o nome

```

```

        "preco": 4200.00,          # Atualizou o preço
        "dimensoes": (35, 9, 21)  # Atualizou as dimensões
    },
    (103, "Acessórios"): {        # Novo produto adicionado
        "nome": "Controle DualSense",
        "preco": 450.00,
        "dimensoes": (16, 11, 6)
    }
}

# Mesclando os dados novos no nosso estoque original
estoque.update(novos_dados)

print("\n--- Estoque Atualizado (.update()) ---")
print(f"Total de produtos diferentes no estoque: {len(estoque)}")
print(f"Novo preço do PS5: R$ {estoque[(101, 'Consoles')][ 'preco' ]}")

```

Busca segura: Produto não encontrado!

--- Relatório de Envios (Usando Desempacotamento) ---

Produto: PlayStation 5 | Categoria: Consoles

Dimensões para frete: 39cm x 10cm x 26cm

Produto: Elden Ring | Categoria: Jogos

Dimensões para frete: 17cm x 13cm x 1cm

--- Estoque Atualizado (.update()) ---

Total de produtos diferentes no estoque: 3

Novo preço do PS5: R\$ 4200.0

```

In [ ]: # O que observar no exemplo acima? Ele nos mostra na prática:
        # A tupla (101, "Consoles") virou uma chave única. Como listas são mu
        # O .update() poupou linhas de código. Em vez de alterar item por ite
        # O código ficou limpo com o desempacotamento: largura, altura, prof

```

```

In [ ]: ##### CONTINUAÇÃO DOS DESAFIOS E PRÁTICAS!

```

```

In [ ]: # Desafio Python3 - Extra - Aplicação de mundo real - Banco Python3
        """
        Para consolidar tudo o que vimos, vamos criar um pequeno Simulador de Cai
        Aqui está o código comentado para o discente estudar a estrutura.

        Dados do arquivo texto
        Significado:
        • horas_estudo → horas estudadas pelo aluno - Francisco
        • frequencia → frequência nas aulas
        • aprovado → 1 = aprovado / 0 = reprovado

        """

```

```

In [ ]: ### # --- Inicialização ---
        saldo = 500.0 # Tipo float (dinheiro)
        nome_usuario = "Francisco- VIP" # Tipo string
        operacao = "" # Variável para controlar o loop

        print(f"--- Bem-vindo ao Python3 Bank, {nome_usuario}! ---")

        # --- Estrutura de Repetição (Loop Principal) ---
        while operacao != "3":
            print("\nEscolha uma opção:")

```

```

print("1 - Ver Saldo")
print("2 - Sacar Dinheiro")
print("3 - Sair")

operacao = input("\nDigite o número da operação: ")

# --- Estrutura Condicional ---
if operacao == "1":
    print(f"Seu saldo atual é: R$ {saldo:.2f}")

elif operacao == "2":
    valor_saque = float(input("Quanto deseja sacar? "))

    # Operador de Comparação e Operador Aritmético
    if valor_saque <= saldo:
        saldo -= valor_saque # Equivalente a: saldo = saldo - valor_
        print(f"Saque de R$ {valor_saque:.2f} realizado com sucesso!")
    else:
        print("Saldo insuficiente para esta operação.")

elif operacao == "3":
    print("Encerrando sistema... Obrigado!")

else:
    print("Opção inválida. Tente novamente.")

print("--- Sessão Finalizada ---")

```

--- Bem-vindo ao Python3 Bank, Francisco- VIP! ---

Escolha uma opção:

- 1 - Ver Saldo
- 2 - Sacar Dinheiro
- 3 - Sair

In [ ]: *# Desafio Python3 - Extra - Aplicação de mundo real com IA (ML) + Ciência*

```

In [8]: # Código Python Completo (IA + ML + Ciência de Dados)
# -----
# Exemplo simples de Data Science + ML
# -----
# Importação das bibliotecas

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# -----
# 1. Ler arquivo CSV
# -----
dados = pd.read_csv("alunos.csv")
# Mostrar os dados
print("Dados carregados:")
print(dados)

# -----
# 2. Separar entradas e saída
# -----
# Variáveis de entrada

```

```
X = dados[["horas_estudo", "frequencia"]]
# Variável alvo
y = dados["aprovado"]

# -----
# 3. Dividir dados de treino e teste
# -----
X_treino, X_teste, y_treino, y_teste = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42
)

# -----
# 4. Criar modelo de Machine Learning
# -----
modelo = DecisionTreeClassifier()
# Treinar modelo
modelo.fit(X_treino, y_treino)

# -----
# 5. Fazer previsões
# -----
previsoes = modelo.predict(X_teste)

# -----
# 6. Avaliar desempenho
# -----
acuracia = accuracy_score(y_teste, previsoes)
print("\nPrevisões:")
print(previsoes)
print("\nAcurácia do modelo:")
print(acuracia)

# -----
# 7. Teste com novo aluno (CORRIGIDO)
# -----
# Não usar uma lista pura. Criar um DataFrame com os nomes exatos das col
novo_aluno = pd.DataFrame([[4, 70]], columns=["horas_estudo", "frequencia"]
# Assim, garantir que o scikit-learn saiba exatamente que o número 4 corr
# o 70 corresponde a "frequencia", eliminando o avisos de segurança (User

# Agora o predict roda sem gerar nenhum UserWarning
resultado = modelo.predict(novo_aluno)

print("\nNovo aluno:")
if resultado[0] == 1:
    print("Aluno provavelmente APROVADO")
else:
    print("Aluno provavelmente REPROVADO")
```

Dados carregados:

|   | horas_estudo | frequencia | aprovado |
|---|--------------|------------|----------|
| 0 | 2            | 60         | 0        |
| 1 | 5            | 75         | 1        |
| 2 | 1            | 50         | 0        |
| 3 | 8            | 90         | 1        |
| 4 | 6            | 80         | 1        |
| 5 | 3            | 65         | 0        |
| 6 | 7            | 85         | 1        |

Previsões:

[0 1 0]

Acurácia do modelo:

1.0

Novo aluno:

Aluno provavelmente APROVADO

In [ ]: